BUBBLE-BLUE: private group messaging over an open Bluetooth mesh, measured on six boards

Paul-Maxence Baraton[1], Nadjib Achir[1], Philippe Jacquet[1], and Wilfred Doré[2]
[1]Inria, France; {paul-maxence.baraton, nadjib.achir, philippe.jacquet}@inria.fr
[2]Qualcomm, France; wdore@qti.qualcomm.com

Abstract. BUBBLE-BLUE is a multi-hop private group that rides on an open Bluetooth Low Energy (BLE) mesh: any node may relay, only key holders may read. We present its design and the first implementation, over BLE L2CAP connection-oriented channels and without pairing, on six single-board computers. Spread through a building on battery, the boards formed a four-hop line of about 60 m on their own, with a median delay of 162 ms at the far end.

1. THE IDEA

Local communication without infrastructure still collapses onto a Wi-Fi access point or a phone operating system that will not host a third-party mesh. Yet the settings that need it most are those where infrastructure is absent, saturated or distrusted: a crowd at an event, a site without coverage, an emergency response. Bluetooth is on every device and needs no operator.

BUBBLE-BLUE [1] separates two things that existing Bluetooth meshes merge. The *topological mesh* is open: any node may join it and relay traffic, including traffic it cannot read. The *bubbles* are private: a group exists only for the holders of its key. A stranger's phone can therefore extend the reach of a conversation it will never see. To keep the cost of flooding low, only an elected backbone, a connected dominating set (CDS), retransmits. The original note gave the algorithms and simulations. This work contributes the BLE transport and frame format, a revocation protocol that reaches members several hops away, and measurements on real boards, in a room and across a building.

2. STATE OF THE ART

Bluetooth meshes and messengers. The SIG Mesh profile [2] is a managed-flooding stack for lighting and sensors; nodes are provisioned, and a relay must hold the network key. BitChat [3] shows that a BLE-only chat can reach consumer scale; it floods over GATT without a backbone. Bridgefy flooded messages among protesters, and its cryptanalysis [4] shows what a mesh used in a crowd must withstand: impersonation, tracking and denial of service. Briar [5] synchronises between *pre-paired* contacts over Classic Bluetooth, Wi-Fi and Tor. Reticulum [6] routes to cryptographic identities over any link.

Backbones for broadcast. Electing a subset of forwarders is classic in ad hoc networks: Wu and Li's marking rule [7] yields a CDS from two-hop knowledge, and multipoint relays [8], the basis of OLSR, reduce flooding from each node's view. BLE makes this matter more than Wi-Fi, because it has no native broadcast on a connection: every local flood is a sequence of unicasts.

Table 1. Where BUBBLE-BLUE sits. "Open relay": a node may forward a frame it cannot decrypt.

System	Data plane	Backbone	Open relay
SIG Mesh [2]	managed flood	provisioned	no
BitChat [3]	GATT flood	none	n/a
Briar [5]	RFCOMM, Wi-Fi, Tor	contacts	no
Reticulum [6]	multi-transport	identities	no
BUBBLE-BLUE	LE CoC	elected CDS	yes

The gap. Table 1 places BUBBLE-BLUE: none of the other systems lets a node sit in an elected backbone and relay for a group it does not belong to.

3. DESIGN

Links without pairing. A node advertises a compact presence record in a connectable BLE advertisement: an anonymous identifier, a relay flag, and the address at which it accepts connections (manufacturer data under company identifier `0xFFFF`, reserved by the Bluetooth SIG for testing, marking the prototype as experimental). Neighbours then open an L2CAP connection-oriented channel at security level *low*: no pairing, no bond, no allow-list. Membership is a 32-byte group key, not a Bluetooth relationship. Periodic HELLO messages fill a neighbour table; a neighbour silent for three periods (15 s) is dropped.

One frame, two planes. Every frame starts with a 26-byte clear header: type, source, destination, TTL, sequence and timestamp. The payload is sealed with AES-256-GCM under a per-packet key, HKDF-SHA256 of the group key with source, sequence and timestamp. All header fields but TTL are authenticated. A relay that does not hold the key can thus decrement TTL and forward the frame; it cannot read the text or forge its sender. A message of up to 470 bytes fits one frame.

Who relays. Each node applies the Wu–Li rule [7] to its two-hop view and relays if it is marked, or if it bridges two links the backbone has not covered. Duplicates are suppressed on (source, sequence), so a flood crosses each relay once. The receiving screen shows the hop count and the neighbour a message came through.

Removing a member. The coordinator, the member with the smallest anonymous identifier, revokes by rotating the group key and flooding it wrapped under pairwise X25519

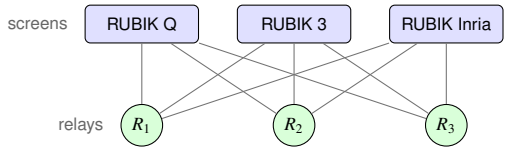


Figure 1. The demonstration stand: every message between two screens crosses one of three relays.

keys, skipping the victim. The victim keeps relaying but can no longer read. Keys of members beyond one hop arrive by a signed ANNOUNCE every 30 s; revoking before then is refused, which defines a join window.

What an adversary can do. A passive radio in range learns who transmits, toward which address, and how far a frame may still go; it does not learn the text. The Bluetooth address is stable and in clear, because that is how a neighbour is redialled without a bond: payload privacy is not location privacy. An active non-member can make the backbone carry junk, reset TTL, which is not authenticated, or pre-empt a (source, sequence) pair at a relay to cause selective loss. A member who controls its address can grind it to become coordinator. These are accepted limits of the prototype.

On real boards. A privileged daemon owns the Bluetooth adapter; touch screens are unprivileged clients. Screens are three Thundercomm RUBIK Pi 3 (Qualcomm QCS6490, Ubuntu 24.04, kernel 6.8, BlueZ 5.72); relays are three Arduino UNO Q (Qualcomm QRB2210, Debian 13, kernel 7.0, BlueZ 5.82). On the tested UNO Q image the controller cannot dial while it is peripheral on any link, so relays always dial and screens never do: every path alternates screen and relay, and on a table the graph is $K_{3,3}$ (Fig. 1). The same kernel leaks one L2CAP signalling identifier per received credit, so links are recycled before the pool of 254 runs out, and a controller that stops answering is reloaded locally without rebooting the board.

Table 2. Measurements on the six boards.

Stand ($K_{3,3}$, one room)	
Messages received by the five other boards	75/75
Rebooted screen reads the group, after daemon start	4.4 s
Relay whose controller was cut reads again, no reboot	130 s
Revoked board reads the next message (2 trials)	no
Four-hop line (building, battery, ∼15 m spacing)	
Messages received four hops away	76/79
Median delay, 1 / 2 / 3 / 4 hops (ms)	85 / 101 / 143 / 162
Last link carried away: held to	∼30 m



Figure 2. The line the boards formed on their own; median delays.

4. RESULTS

On the stand. Every message reached the five other boards (Table 2). Joining is dominated by the boot: a rebooted screen read the group 4.4 s after its daemon started. AES-GCM (30k–173k packets/s) is far above the link; the bottleneck is the relay's Bluetooth stack, near 10 msg/s.

Across a building. With the boards about 15 m apart and one relay off, the mesh formed a four-hop line by itself (Fig. 2). From on-board logs (clocks within 50 ms), the far end received 76 of 79 messages, missing only the three sent while the line formed; each hop adds a few tens of milliseconds. Carried away, the far screen's link held to about 30 m, dropped, and returned within seconds of the board coming back: mobility appears as ordinary link churn. When the sender's only relay lost power, the line was dark for 95 s: a line has no detour.

In simulation and next. On a unit-disk radio the backbone adds no path stretch, and TTL 10 spans about twenty nodes (a simulation result). A day-long battery run is under way; the link logs form a contact graph on which to replay the protocol under real mobility traces.

Acknowledgment. Boards from the Qualcomm University Platforms program. AI-assisted tools were used for software, analysis and editing; the authors checked all results.

REFERENCES

[1] N. Achir and P. Jacquet, "BUBBLE-BLUE a multihop private network based on Bluetooth," *arXiv preprint arXiv:2509.00967*, 2025.

[2] *Mesh Profile Specification, Version 1.0*, Bluetooth SIG, 2017.

[3] J. Dorsey *et al.*, "BitChat: Bluetooth mesh chat," https://github.com/permissionlesstech/bitchat, 2025.

[4] M. R. Albrecht, J. Blasco, R. B. Jensen, and L. Marečková, "Mesh messaging in large-scale protests: Breaking Bridgefy," in *Topics in Cryptology – CT-RSA 2021*, ser. LNCS, vol. 12704. Springer, 2021, pp. 375–398.

[5] The Briar Project, "Briar," https://briarproject.org/, 2024.

[6] M. Qvist, "Reticulum / LXMF," https://github.com/markqvist/Reticulum, 2024.

[7] J. Wu and H. Li, "On calculating connected dominating set for efficient routing in ad hoc wireless networks," in *Proc. 3rd Int. Workshop on Discrete Algorithms and Methods for Mobile Computing and Communications (DIAL-M)*, 1999, pp. 7–14.

[8] A. Qayyum, L. Viennot, and A. Laouiti, "Multipoint relaying for flooding broadcast messages in mobile wireless networks," in *Proc. 35th Hawaii Int. Conf. on System Sciences (HICSS)*. IEEE, 2002, pp. 3866–3875.